COLLEGE OF SCIENCE AND TECHNOLOGY

CE80763 REAL-TIME SYSTEMS

UNIT 2: MINI PROJECTS REPORT

Mini Projects 2.1, 2.2 and 2.3 | Analysis of the supplied implementations

GROUP 1 MEMBERS

1. MUHIRE VALENTIN 223026677

2.RUBIMBURA DAVID 221014566

3.IRADUKUNDA REOBOTH 223026686

Executive Summary

This report addresses the three practical assignments in the CE80763 Real-Time Systems Unit 2 Lab Manual. It analyses the supplied implementations of (1) RM, EDF, response-time analysis and priority inheritance; (2) an RM-versus-EDF schedulability comparison tool; and (3) partitioned multiprocessor scheduling using FF, FFD, BFD and WF.

The worked examples and task sets are interpreted using the task parameters in the supplied Python code. The report distinguishes utilization-bound results from exact schedulability conclusions and identifies a few places where the implementation's output does not fully meet the lab manual's requested evidence. The code is treated as the implementation under review; no claim is made that every plot or full-hyperperiod simulation was executed as part of preparing this report.

1. Aim and Task Model

The aim is to demonstrate how periodic real-time tasks can be scheduled on a single processor and partitioned across multiple cores. Each task is represented by its name, worst-case execution time C, period T, and, in Projects 2.1 and 2.2, relative deadline D. When D is omitted, the code sets D = T, representing implicit deadlines. Task utilization is u = C/T, and total utilization is $U = \Sigma(C_i/T_i)$.

Rate Monotonic (RM) assigns fixed priorities according to period: a shorter period receives higher priority. When deadlines are constrained, Deadline Monotonic (DM) assigns higher priority to the task with the shorter relative deadline. Earliest Deadline First (EDF) dynamically chooses the ready job with the earliest absolute deadline. The Liu–Layland RM utilization bound is $U \leq n(2^{(1/n)} - 1)$, a sufficient test for the classical independent periodic implicit-deadline task model. EDF's $U \leq 1$ test is exact for that same uniprocessor implicit-deadline model.

2. Mini Project 2.1 — RM, EDF and Priority Inheritance

Objective and procedure. The project models the three-task worked example, adds the fourth task from Section 2.3.2, simulates RM and EDF over the 24-unit hyperperiod, performs response-time analysis, and illustrates priority inversion with and without PIP.

Worked example 1. The task set is τ_1=(C=1,T=4), τ_2=(2,6), and τ_3=(2,12). Its utilization is 1/4 + 2/6 + 2/12 = 0.750. The three-task RM bound is $3(2^{(1/3)}-1) \approx 0.780$. Because $0.750 \leq 0.780$, the sufficient RM test passes.

Worked example 2. Adding τ_4=(C=1,T=8) gives U = 0.875. The four-task RM bound is $4(2^{(1/4)}-1) \approx 0.757$, so the sufficient bound no longer proves schedulability. This is not itself proof of failure. The RTA results from the supplied task set are R_1=1, R_2=3, R_4=4, and R_3=11 time units; each is within its implicit deadline. Thus RTA establishes RM schedulability for this set even though the sufficient utilization test fails. EDF also passes the implicit-deadline utilization test because $U=0.875 \leq 1$.

Scheduling plots. The two-panel plot places RM above EDF on a shared time axis. RM always favours the shortest-period ready task, while EDF favours the ready job with the nearest absolute deadline. The plots should be used to illustrate policy choices; the timeline functions do not explicitly record each job's completion time or report deadline misses.

Priority inversion and PIP. In the demonstration, L acquires resource R at t=0 and needs six execution ticks. H arrives at t=1 and requests R. M arrives at t=2 and needs four ticks without the resource. Without PIP, M can run ahead of L while H remains blocked. With PIP, L inherits H's effective priority while H is waiting, so L can finish its critical section without being pre-empted by M. In the supplied simplified schedule, H first runs at approximately tick 10 without PIP and tick 6 with PIP. The exact blocking-time metric should be measured from H's arrival until resource release; the code currently prints H's first run start and run duration rather than directly tabulating that blocking interval. Therefore the requested blocking-time comparison still needs to be explicitly calculated in the submitted evidence.

Lab-task extensions. The manual also suggests DM for constrained deadlines, plotting the RM bound for n=1…10 against the EDF bound of 1, implementing Least Laxity First and counting context switches, and adding another medium-priority task. These extensions are not present in the supplied Project 2.1 code and should not be claimed as completed unless implemented separately.

3. Mini Project 2.2 — RM versus EDF Schedulability Comparison

Objective and approach. The tool reports each task's utilization, total U, the RM bound, an RM verdict, an EDF verdict and, when the RM bound fails, response-time analysis. It can accept task sets in Python lists or load them from CSV. It optionally plots RM/DM and EDF schedules over a capped horizon.

Results across the supplied task sets are summarized below. The utilization values are calculated from the parameters in the code; RTA values are response-time calculations using the fixed-priority recurrence. The RM bound is sufficient, so a failed bound test must lead to further analysis rather than an automatic unschedulable verdict.

Task set 1 — three-task worked example: U=0.750; RM bound≈0.780; the RM bound passes; EDF passes because U≤1.

Task set 2 — four-task worked example: U=0.875; RM bound≈0.757; the RM bound fails, but RTA gives R_1=1, R_2=3, R_4=4 and R_3=11, all meeting their deadlines; EDF passes.

Task set 3 — boundary set: (1,4), (2,6), (1,10), (1,11). U≈0.774; the four-task RM bound≈0.757. The sufficient test fails, but RTA gives response times of 1, 3, 4 and 6 time units in priority order, all within their deadlines; EDF passes.

Task set 4 — overloaded set: (3,5), (3,6). U=0.600+0.500=1.100. Since demand exceeds one processor's capacity, the implicit-deadline set is infeasible under both RM and EDF.

Task set 5 — constrained-deadline set: τ_1=(3,6,D=6), τ_2=(2,8,D=3). U≈0.750. DM order places τ_2 above τ_1; RTA gives R_2=2≤3 and R_1=5≤6. The set is schedulable under this fixed-priority analysis. However, the code's early RM-bound PASS path uses the implicit-deadline bound even when a constrained deadline exists, so that verdict is not justified by the bound alone.

Task set 6 — ten tasks with C=1 and periods 51 through 60: U≈0.181; the ten-task RM bound≈0.718; the sufficient test passes. This illustrates how the bound decreases toward ln(2)≈0.693 as task count grows, although the supplied script does not plot the full bound sweep requested in the lab tasks.

Reflection (RM bound versus RTA). The four-task worked example is the clearest illustration of why sufficient and exact analyses must not be confused. Its utilization, 0.875, exceeds the four-task Liu–Layland bound of approximately 0.757. The bound therefore cannot guarantee RM schedulability, but it does not declare the task set impossible. RTA accounts for the interference that higher-priority tasks can cause to each task. For the set, the calculated response times are 1 for τ_1, 3 for τ_2, 4 for τ_4 and 11 for τ_3. All are no greater than their deadlines of 4, 6, 8 and 12 respectively. Therefore the set is schedulable under RM according to RTA, and EDF also passes because U≤1. This demonstrates the value of applying a more precise analysis whenever the utilization-bound test is inconclusive.

4. Mini Project 2.3 — Partitioned Multiprocessor Scheduling

Objective and algorithms. The project assigns tasks to four cores using a per-core RM admission test. A proposed assignment is accepted only if the total utilization of the resulting core task set does not exceed rm_bound(n), recomputed for the number of tasks on that core. FF places tasks in input order on the first admissible core. FFD sorts tasks by decreasing utilization before using FF. BFD sorts by decreasing utilization and chooses the admissible core with the least calculated remaining bound capacity. WF chooses the admissible core with the most remaining bound capacity.

Comparison of the supplied test sets (m=4). Each successful row reports the number of non-empty cores and their utilization. The table below is calculated by applying the algorithms in the supplied code.

Balanced set: ten tasks each with u=0.05, total U=0.500. FF, FFD and BFD pack all ten tasks onto one core with U_core=0.500; WF distributes them across four cores with loads 0.15, 0.15, 0.10 and 0.10. All four heuristics succeed.

Skewed set: tasks have utilizations 5/12≈0.417, 5/13≈0.385, 0.050, 0.040 and 1/30≈0.033, for total U≈0.925. FF, FFD and BFD use two cores with loads approximately 0.801 and 0.123. WF uses four cores with loads approximately 0.417, 0.385, 0.050 and 0.073. All four succeed, but their packing efficiency differs.

Adversarial set: four tasks each have C=6 and T=11, so each utilization is approximately 0.545 and total U≈2.182. Every heuristic assigns one task to each of the four cores. No pair can share a core under the implemented two-task RM bound because 2×0.545≈1.091 exceeds the two-task bound≈0.828. This illustrates fragmentation and the need for one core per task, but it does not cause a partition failure because exactly four tasks are supplied for four cores. To meet the manual's stronger requirement of an adversarial set that actually defeats at least one heuristic within m=4, the task set needs an additional task or a different mix of task sizes.

Tight set: eight tasks have C=2 and periods 11 through 18, giving total U≈1.132. FF, FFD and BFD use two cores with loads approximately 0.645 and 0.487. WF uses four cores with loads approximately 0.293, 0.284, 0.279 and 0.276. All methods succeed, but WF uses more cores.

Analysis of FFD versus FF and WF. FFD can improve over FF when input order is unfavourable because it places the largest-utilization tasks first, reducing the chance that small tasks consume capacity needed by larger tasks. BFD tries to fill an admissible core more tightly, while WF deliberately spreads tasks onto the core with the most remaining capacity. The supplied results show that FFD is not always better: on the balanced and tight sets, FF/FFD/BFD use fewer cores than WF; on the skewed set, WF spreads load across more cores. These are heuristic outcomes, not proofs of optimality. The admission test is a sufficient RM bound, so a task may be rejected even when a more exact analysis would show that the core is schedulable.

The code's final chart uses the exact hyperperiod for periods 11 through 18. That least common multiple is very large, so simulating and drawing every tick can be slow and memory-intensive. A bounded chart horizon (as permitted in the manual's suggested approach) is more practical for demonstrating per-core behavior.

5. Overall Evaluation and Recommendations

The three programs cover the main ideas required by the manual: fixed and dynamic priority scheduling, response-time analysis, priority inheritance, utilization-bound tests, and multiprocessor bin-packing heuristics. Their core logic is suitable for an educational demonstration, but some required deliverables need additional evidence or qualification.

Before final submission, record the actual console output and export the requested plots. For Project 2.1, explicitly calculate H's blocking interval rather than reporting only H's first run. For Project 2.2, ensure constrained-deadline sets do not receive an implicit-deadline RM-bound verdict without the appropriate analysis, and label EDF's U≤1 verdict as exact only for implicit deadlines. For Project 2.3, replace or extend the current four-task adversarial set so that a heuristic genuinely fails to place all tasks within four cores, and cap the plot horizon for the tight set.

These are alignment and interpretation recommendations based on comparing the supplied source code with the lab manual. They do not imply that the programs have been executed in this report-generation process.

Conclusion

The Unit 2 mini projects provide a progression from single-processor scheduling to multiprocessor assignment. Project 2.1 shows how RM, EDF, RTA and PIP address different scheduling questions. Project 2.2 demonstrates that a sufficient utilization bound can fail to prove schedulability even when RTA succeeds. Project 2.3 illustrates that partitioning depends on both aggregate utilization and the distribution of individual task utilizations.

The strongest final submission will pair the analysis in this report with actual program output and readable Gantt charts, and will clearly state the assumptions behind each schedulability conclusion.

Appendix A — Key Equations

Utilization: $u_i = C_i/T_i$; total utilization: $U = \Sigma u_i$.

RM sufficient bound: $U \leq n(2^{(1/n)}-1)$.

Fixed-priority response-time recurrence: $R_i = C_i + \Sigma \text{ ceil}(R_i/T_j)C_j$, over higher-priority tasks j.

Implicit-deadline uniprocessor EDF test: $U \leq 1$.

Hyperperiod: $H = \text{lcm}(T_1, T_2, \ldots, T_n)$.